

Unicom TIC Management System

1. Introduction

The **Unicom TIC Management System (UMS)** is a simple desktop application designed for beginners to manage basic operations of a school. It handles courses, subjects, students, exams, marks, and timetables, with a **login system** for **Admin, Staff, Students, and Lecturers**. The system includes **computer labs and lecture halls allocation** to assign places (labs for computer classes, halls for regular lessons) to timetable entries. You'll build it using **C# WinForms** with a **Model-ViewController (MVC)** structure and a **SQLite database** to save data.

This assignment helps you learn basic C# programming, create forms, use a database, and build a login system. It's designed to be easy, focusing on simple tasks to help you understand coding for a school system.

2. Objectives

- Build a simple desktop app using C# WinForms and a basic MVC design.
- Create modules: Course & Subject Management, Student Management, Exam & Marks Management, and Timetable Management with Computer Labs and Lecture Halls Allocation.
- Make a basic login system for Admin, Staff, Students, and Lecturers with role-based access.
- Use MVC to organize data (Model), forms (View), and logic (Controller).
- Save data in a SQLite database with simple add, view, edit, and delete actions.
- Build an easy WinForms interface with buttons and lists.
- Check inputs and show error messages.
- Use C# async/await to keep the app smooth.

3. System Scope and Features

The UMS has a few simple modules, with data saved in SQLite and access controlled by a login system. Computer labs and lecture halls are part of the Timetable module.

3.1 Login System

- Make a login form where **Admin, Staff, Students, and Lecturers** enter a username and password.
- Save usernames, passwords, and roles in a SQLite Users table (use plain text passwords, no hashing).
- Set role-based access:

- **Admin:** Can add, edit, delete, and view everything (courses, students, exams, timetables).
- **Staff:** Can view timetables (with labs/halls) and add/edit exams and marks. ○
Student: Can view their own marks and timetable (with labs/halls).
- **Lecturer:** Can view timetables (with labs/halls) and add/edit marks.
- Show a main dashboard after login with buttons for allowed features based on the role.

3.2 Course & Subject Management

- Manage courses: Course ID, Course Name (e.g., “BSc Computer Science”).
- Manage subjects: Subject ID, Subject Name, Course ID (each subject belongs to one course).
- **Admin:** ○ Add a course (e.g., “BSc Computer Science”). ○ Add a subject to a course (e.g., “Programming”). ○ View courses and subjects in a list (DataGridView).
 - Edit or delete courses and subjects.
- **All roles:** View courses and subjects (read-only).

3.3 Student Management

- Manage students: Student ID, Name, Course ID (each student is in one course).
- **Admin:**
 - Add a student. ○ View students in a list.
 - Edit or delete student details.
- **Students:** View their own details (e.g., Name, Course) in a form.

3.4 Exam & Marks Management

- Manage exams: Exam ID, Subject ID, Exam Name (e.g., “Midterm”).
- Manage marks: Student ID, Exam ID, Score (0–100).
- **Admin:** ○ Add an exam for a subject.
 - View exams in a list.
- **Admin/Lecturers:**
 - Add or edit marks for students.
 - View marks in a list.
- **Students:** View their own marks.
- **Staff:** View marks and add/edit exams.

3.5 Timetable Management with Computer Labs and Lecture Halls Allocation

- **Purpose:** Schedule classes, assigning a **computer lab** (e.g., for “Programming”) or **lecture hall** (e.g., for “Math”) to each class.
- **Entities:**
 - **Timetable:** Timetable ID, Subject ID, Time Slot (e.g., “Monday 10 AM”), Room ID.
 - **Room:** A computer lab (e.g., “Lab 1”) or lecture hall (e.g., “Hall A”), with Room ID, Room Name, Room Type (“Lab” or “Hall”).
- **Functionality:**
 - **Admin:**
 - ✦ Add a timetable entry by picking a subject, time slot, and room (lab or hall) from a dropdown (ComboBox).
 - ✦ View, edit, or delete timetable entries in a list, showing the room.
 - ✦ Example: “Programming, Monday 10 AM, Lab 1” or “Math, Tuesday 2 PM, Hall A”.
 - **All roles:**
 - ✦ View the timetable in a list, showing Subject, Time Slot, and Room.
- **Note:** You don’t need to check for conflicts (e.g., same room booked twice is okay) to keep it simple.
- **Example:**
 - Admin adds: “Database Management, Tuesday 2 PM, Lab 2” (computer lab).
 - Student sees: “Database Management, Tuesday 2 PM, Lab 2” in their timetable.

4. Architecture and Design

4.1 MVC Architecture

- **Model:** Classes for data (e.g., Course, Student, Room) with simple properties.
- **View:** WinForms forms for login, dashboard, and modules, using buttons and lists.
- **Controller:** Handles button clicks, saves data, and updates forms.

4.2 Data Relationships

- **Course-Subject:** One-to-many; a course has multiple subjects.
- **Student-Course:** Many-to-one; multiple students in one course.

- **Exam-Subject:** One-to-many; a subject has multiple exams.
- **Mark-Student and Mark-Exam:** Links a student and exam with a score.
- **Timetable-Subject:** Each timetable entry is for one subject and a room (lab or hall).
- **Room:** Stores computer labs and lecture halls (e.g., "Lab 1," "Hall A").
- **User:** Stores login details.

4.3 SQLite Database

- Use SQLite to save data in simple tables. Example:
 - Users(UserID INTEGER PRIMARY KEY, Username TEXT, Password TEXT, Role TEXT)
 - ✦ Example: (1, "admin1", "pass123", "Admin")
 - Courses(CourseID INTEGER PRIMARY KEY, CourseName TEXT)
 - Subjects(SubjectID INTEGER PRIMARY KEY, SubjectName TEXT, CourseID INTEGER, FOREIGN KEY(CourseID) REFERENCES Courses)
 - Students(StudentID INTEGER PRIMARY KEY, Name TEXT, CourseID INTEGER, FOREIGN KEY(CourseID) REFERENCES Courses)
 - Exams(ExamID INTEGER PRIMARY KEY, ExamName TEXT, SubjectID INTEGER, FOREIGN KEY(SubjectID) REFERENCES Subjects)
 - Marks(MarkID INTEGER PRIMARY KEY, StudentID INTEGER, ExamID INTEGER, Score INTEGER, FOREIGN KEY(StudentID) REFERENCES Students, FOREIGN KEY(ExamID) REFERENCES Exams)
 - Rooms(RoomID INTEGER PRIMARY KEY, RoomName TEXT, RoomType TEXT)
 - ✦ Example: (1, "Lab 1", "Lab"), (2, "Lab 2", "Lab"), (3, "Hall A", "Hall"), (4, "Hall B", "Hall")
 - Timetables(TimetableID INTEGER PRIMARY KEY, SubjectID INTEGER, TimeSlot TEXT, RoomID INTEGER, FOREIGN KEY(SubjectID) REFERENCES Subjects, FOREIGN KEY(RoomID) REFERENCES Rooms)
- Use System.Data.SQLite to add, view, edit, and delete data.

4.4 Role-Based Access

- Check username and password in a login form to show the dashboard.
- Limit access by role:
 - Admin: Use all forms to add/edit/delete.
 - Staff: View timetables, add/edit exams/marks.

- Student: View marks and timetable.
- Lecturer: View timetables, add/edit marks.
- Hide buttons for actions a role can't do (e.g., Students can't see "Add Course").

4.5 Computer Labs and Lecture Halls Allocation

- Save a small list of rooms (e.g., "Lab 1," "Lab 2," "Hall A," "Hall B") in the Rooms table.
- Admin picks a room (lab or hall) from a dropdown when adding a timetable entry.
- Show the room in the timetable list for all users (e.g., "Programming, Lab 1").
- Show an error if no room is picked (e.g., "Please select a room").

5. Technical Requirements

- **Environment:** Visual Studio with C# WinForms.
- **Database:** SQLite, using System.Data.SQLite (install via NuGet).
- **UI:** WinForms with Form Designer, using Button, TextBox, ComboBox, DataGridView.
- **OOP:**
 - **Encapsulation:** Use private fields and public properties (e.g., private string name; with public string Name { get; set; }).
 - **Inheritance** (optional): Use a Person class for Student if you want to share Name.
- **Design Pattern:**
 - **MVC:** Keep data, forms, and logic separate.
 - **Singleton** (optional): Use one DatabaseManager for SQLite connections.
- **C#:**
 - **Async/Await:** Use for database tasks to keep forms responsive (e.g., async save).
- **Error Handling:**
 - Check inputs (e.g., Course Name not empty, Score 0–100). ○ Show messages (e.g., "Wrong password" or "Pick a room").
 - Handle database errors (e.g., "Can't connect to database").

6. Folder Structure

Keep it simple:

UnicomTICManagementSystem (Project Name)

Models

Course.cs

Subject.cs

Student.cs

Exam.cs

Mark.cs

Timetable.cs

Room.cs

User.cs

Views

LoginForm.cs

MainForm.cs

CourseForm.cs

StudentForm.cs

ExamForm.cs

MarkForm.cs

TimetableForm.cs

Controllers

LoginController.cs

CourseController.cs

StudentController.cs

ExamController.cs

MarkController.cs

TimetableController.cs

Repositories

DatabaseManager.cs

Program.cs

7. Development Process

Follow these steps:

1. Setup:

- Create a C# WinForms project in Visual Studio. ○ Install System.Data.SQLite via NuGet.
- Make the folder structure.

2. Database:

- Create unicomtic.db with tables for Users, Courses, Subjects, Students, Exams, Marks, Rooms, Timetables.
- Add code in DatabaseManager to create tables if they don't exist.

3. Models:

- Make classes (e.g., Room with RoomID, RoomName, RoomType) with simple properties.

4. Repository:

- Create DatabaseManager with async methods to add, view, edit, delete data (e.g., AddRoom).

5. Views:

- Use Form Designer to make:
 - ✦ LoginForm: TextBox for username/password, login button.
 - ✦ MainForm: Dashboard with buttons (e.g., "Courses" for Admin).
 - ✦ Module forms (e.g., TimetableForm): ComboBox for rooms, DataGridView for lists, buttons for add/edit.
- Add clear labels (e.g., "Pick Room: Lab 1, Hall A").

6. Controllers:

- Make controllers to handle button clicks (e.g., `TimetableController.AddTimetable`).
- Check login with `LoginController`.

7. Test:

- Test login for all roles. ○ Test adding a course, student, exam, mark, and timetable with a lab or hall. ○ Check only allowed roles see buttons (e.g., Student can't add courses).
- Test error messages (e.g., empty inputs).

8. Challenges

- **Login:** Matching usernames/passwords and showing right buttons.
- **Roles:** Hiding buttons for wrong roles (e.g., Student can't add exams).
- **Rooms:** Letting Admin pick a lab or hall for timetables.
- **Database:** Setting up tables and saving data.
- **Forms:** Moving between forms without errors.

9. Bonus Features (Optional)

Try these for extra points:

- **Room Filter:** Show only labs or halls in the timetable dropdown.
- **Top Marks:** Show top 3 students by exam marks.
- **Messages:** Show pop-ups (e.g., "Timetable saved!").
- **Error Log:** Save errors (e.g., "Wrong password") to a text file.

10. Deliverables

- **Source Code:** All files and `unicomtic.db`.
- **Documentation:** 1-page report explaining:
 - How your app works (e.g., login, rooms). ○ Problems and fixes.
 - Bonus features, if any.

- **Demo:** Show your app in class, logging in, adding a timetable with a lab/hall, viewing data.

11. Resources

- **Visual Studio:** Use Form Designer, NuGet for System.Data.SQLite.
- **SQLite:** Check System.Data.SQLite tutorials for beginners.
- **WinForms:** See Microsoft's WinForms guides for buttons/lists.
- **C#:** Use Microsoft's C# tutorials for classes, async.
- **MVC:** Search "simple MVC WinForms C#" for examples.

12. Notes

- Start early to learn WinForms and SQLite slowly.
- Test login first to check roles.
- Ask for help with database or forms.
- Keep forms simple with clear buttons.
- Add comments to explain your code.
- Save unicomtic.db often.